

An On-Chain Settlement Layer for Peer-to-Peer Energy Trading under Thailand’s Enhanced Single Buyer Model

GridTokenX: Solana/Anchor Programs and Runtime Architecture

Abstract — The growing adoption of renewable energy has yielded participants with surplus generation capacity who currently cannot engage in peer-to-peer (P2P) trading under Thailand’s Enhanced Single Buyer model. This article presents an on-chain settlement layer for P2P energy trading in which transaction settlement executes on a Solana cluster. The system features a consortium governance structure mapped to Thailand’s grid operators (EGAT, MEA, PEA), an application layer implemented as a set of Anchor smart-contract programs, and consensus and finality driven by Solana’s PoH/Tower-BFT engine. The market design incorporates a hybrid continuous double auction (CDA) that matches the on-chain order book in real time for immediate trades, such as BESS reserve dispatch, alongside a uniform-price auction that establishes a single clearing price over 15-minute intervals. We evaluate the smart-contract programs using a transaction-level benchmark on a single-node validator (Apple M2, Agave 3.1.10). Across a TPC-C concurrency sweep (500 transactions, 50% NewOrder / 50% Payment), all transactions confirmed with zero failures. Throughput scaled super-linearly with in-flight concurrency, reaching $\approx 74 \text{ tx}\cdot\text{s}^{-1}$ at a concurrency of 40 with no saturation knee across the swept range. Compute cost remained flat at $\approx 23 \text{ k CU}\cdot\text{tx}^{-1}$ across all levels, indicating that the performance ceiling is dictated by single-node block production and write-lock serialisation rather than on-chain execution. That serialisation dominates any path funnelled through a single write-locked account: single-signer mint issuance holds at only $\approx 5.33 \text{ mint}\cdot\text{s}^{-1}$ regardless of device count, and single-fee-payer settlement at $\approx 0.6 \text{ tx}\cdot\text{s}^{-1}$. We identify multi-signer fee-payer pooling, together with sharded collector accounts, as the primary lever for issuance- and settlement-side throughput.

Keywords — peer-to-peer energy trading; blockchain settlement; Solana; continuous double auction; uniform-price auction; Thailand Enhanced Single Buyer.

1. Introduction

Peer-to-peer (P2P) energy trading enables prosumers — households and firms that both produce and consume electricity — to settle surplus generation directly with one another rather than solely through a utility. In Thailand this exchange is currently foreclosed: under the Enhanced Single Buyer (ESB) model all wholesale electricity is procured through a single national off-taker, leaving prosumers with surplus generation no direct route to trade with one another. Realising such an exchange on a public ledger raises three requirements that a general-purpose blockchain does not satisfy by default: (i) **throughput**, because metering telemetry and order flow arrive continuously from many independent devices; (ii) **settlement integrity**, because a trade must be provably authorised by both counterparties yet cannot be allowed to replay; and (iii) **physical backing**, because a token that claims to represent a kilowatt-hour must be tied to an attested, real-world measurement rather than minted at will.

This paper describes GridTokenX, an on-chain energy-trading platform built as a set of Anchor programs on a permissioned Proof-of-Authority (PoA) Solana cluster. The design exploits the Solana Virtual Machine (SVM) execution model directly: it partitions all hot-path state into per-entity program-derived accounts (PDAs) so that unrelated meters, orders, and registrations never contend for the same write lock and therefore execute in parallel under Sealevel. The market runs a hybrid double auction: a continuous double auction (CDA) maintains an on-chain order book and matches compatible orders in real time (`sharded_match_orders`) for immediate trades, such as battery-energy-storage reserve dispatch, while a uniform-price auction clears accumulated bids at a single price over 15-minute epochs coordinated by the oracle’s market-clearing trigger. Matches produced by the off-chain engine settle on-chain through native Ed25519 signature verification and per-order replay nullifiers, so the chain never holds custody of the matching engine yet still enforces authorisation and single-use settlement. Token minting is gated behind a registered set of Renewable Energy Certificate (REC) validators, binding issuance to attested generation. An accompanying treasury program provides a Thai-baht-pegged stablecoin (THBG) with reserve-attested peg invariants and a MasterChef-style staking accumulator, giving the market a baht-denominated settlement unit.

Governance follows a permissioned consortium model. The platform authority named in the governance program is intended to be held jointly by Thailand’s grid operators — the Electricity Generating Authority (EGAT), the Metropolitan Electricity Authority (MEA), and the Provincial Electricity Authority (PEA) — under a two-step authority-transfer discipline (§6.2), so that protocol administration rests with the same institutions that operate the physical grid, while block production runs on a permissioned Proof-of-Authority validator set (§8).

The contributions of this work are:

1. A Sealevel-parallel account architecture in which every high-frequency write — meter telemetry, order placement, settlement — targets a per-entity PDA, with global aggregates kept deliberately stale and reconciled by periodic admin instructions (§4.2).
2. An off-chain-matched, on-chain-settled trade protocol that verifies buyer and seller Ed25519 signatures through instruction-sysvar introspection and prevents replay with per-order nullifier PDAs (§6.3).
3. A REC-validator gating scheme that ties token minting to attested physical energy (§6.4), together with a two-step authority-transfer mechanism for application-layer PoA governance (§6.2).
4. A treasury design with formally stated peg and collateral invariants and a precise, integer-only economic model for price formation, settlement, fees, wheeling charges, and staking rewards (§11).

The remainder of the paper is organised as follows. Section 2 states the methodology — the design principles, the implementation stack, and the evaluation approach. Section 3 presents the execution stack from off-chain services down to the deployed programs. Section 4 develops the SVM execution model — the account/lock discipline, the compute budget, and zero-copy state. Section 5 describes cross-program invocation between the programs. Section 6 sets out the security and trust model. Section 7 traces an end-to-end market cycle, and Section 8 covers the consensus and validator topology. Section 9 reports the key empirical results and discusses their implications. Section 10 is a reading map to the wider documentation, and Section 11 gives the formal price-formation, settlement, and economic equations, each cited to its implementing source line.

2. Methodology

This work follows a **design-and-measure** method: the platform is constructed as a set of on-chain programs whose structure is dictated by the execution semantics of the target runtime, and every architectural claim is grounded in the implementing source and, where behavioural, in a reproducible measurement. This section states the design principles that constrain the implementation (§2.1), the implementation stack itself (§2.2), and the evaluation harnesses used to verify functional correctness and quantify runtime cost (§2.3).

2.1 Design principles

The design is governed by four principles, each a direct consequence of the SVM account model rather than a stylistic preference.

1. **Execution-model-first partitioning.** Because the runtime serialises any two transactions that write the same account, all high-frequency state is placed in per-entity PDAs, and shared configuration is treated as read-only on hot paths. Parallelism is therefore a property of the data layout, not of an added concurrency mechanism (developed in §4.2).
2. **State in accounts, logic in stateless programs.** Programs hold no mutable state; all protocol state lives in program-owned accounts, which keeps authorisation reducible to the runtime ownership rule.
3. **Defensive integer arithmetic.** All value-bearing computations use checked or saturating integer operations over u128 intermediates, and every program enables overflow-checks = true in its release profile, so an arithmetic overflow aborts the transaction rather than wrapping silently.
4. **Cite, do not assert.** Each architectural statement in this paper is anchored to a specific source location (path:line); a claim without such an anchor is treated as a hypothesis rather than a result.

2.2 Implementation

The platform is implemented in Rust and compiled to SBF bytecode with the Anchor framework (anchor-lang and anchor-spl version 1.0.0). It comprises five core programs — energy-token, governance, oracle, registry, and trading — a treasury program providing the pegged settlement unit, and two benchmark crates (blockbench and tpc-benchmark); shared functionality is factored into core and compute-debug crates. Each program is an independent crate rather than a member of a single workspace, and the canonical program identifiers are declared in Anchor.toml [programs.localnet]. Fungible energy and Renewable Energy Certificate balances are held in Token-2022 mints. State structs are declared zero-copy ([account(zero_copy)] #[repr(C)]) and accessed in place through AccountLoader, avoiding full deserialisation of large accounts. Compute-unit consumption is profiled non-invasively by wrapping each handler body in the compute_fn! macro (shared/compute-debug/src/lib.rs:78), which reports remaining budget on localnet and compiles to a no-op in release builds.

2.3 Evaluation approach

Functional correctness and runtime cost are assessed with a layered set of harnesses, chosen so that most properties can be verified without a validator. In-process suites built on LiteSVM exercise program guards and profile compute-unit cost, with the profiling suites asserting that each instruction remains below the 200,000-CU default budget. Integration suites run under a live validator (anchor test, or the standalone runner) to confirm cross-program invocation and settlement end to end, and a Surfpool-based simulation reproduces a mainnet-like environment without a local validator. Throughput and cost under load are characterised with a Criterion micro-benchmark of the matching engine and with the BlockBench and TPC-C workloads; the canonical results are recorded in [BENCHMARKS.md](#). A methodological caveat applies to all on-chain measurements: the development topology is a single-node localnet, so reported figures characterise SVM and runtime behaviour (account locking, compute metering, instruction cost) and not consensus throughput, which would require a multi-validator deployment (§8.3).

3. The Execution Stack, Top to Bottom

Off-chain platform (superproject) IAM / Trading / Oracle services → Chain Bridge (only RPC client) → JSON-RPC / gRPC
Validator node (solana-test-validator on localnet) TPU ingest → SigVerify → Banking stage → PoH → ledger
SVM runtime (Sealevel) Account locks → parallel scheduling → SBF (eBPF-derived) bytecode execution → compute-unit metering
GridTokenX programs (this repo) energy-token governance oracle registry trading + CPI between them, + SPL Token / Token-2022 programs

Key boundary rule (platform-wide, see superproject docs): **no service holds a Solana RPC connection except Chain Bridge**. Everything below the top layer is what this document covers.

4. SVM Execution Model

4.1 Programs are stateless; state lives in accounts

Each program in `programs/*` compiles to SBF bytecode (`anchor build → target/deploy/*.so`) and is deployed at a fixed address (`declare_id!`). A program owns accounts; only the owning program may mutate an account's data. All protocol state — token supply, meter readings, orders, governance config — lives in accounts, not in the programs.

4.2 Transactions declare their account set in advance

A Solana transaction lists every account it will touch and whether each is read-only or writable. The runtime uses this to take **per-account read/write locks** before execution. Two transactions that touch disjoint writable account sets execute **in parallel** on different cores (Sealevel). Two transactions that both write the same account serialise.

This is the most fundamental constraint on the repository's design. Every hot-path write goes to a **per-entity PDA** so that unrelated users/meters/orders never contend:

Hot path	Per-entity account	Seeds	Where
Meter telemetry	MeterState	[b"meter", meter_id]	<code>programs/oracle/src/lib.rs:473</code>
Order placement	Order	[b"order", authority, order_id]	<code>programs/trading/src/lib.rs:1435</code>
Settlement replay guard	OrderNullifier	[b>nullifier", user, order_id]	<code>.../settle_offchain.rs:112</code>
User registration counters	RegistryShard × 16	[b"registry_shard", shard_id]	<code>programs/registry/src/lib.rs:770</code>

Hot path	Per-entity account	Seeds	Where
Settlement escrow	escrow PDA	[b"escrow", user, currency_mint]	.../settle_offchain.rs:140

Shard selection is deterministic from the signer: `key.to_bytes()[0] % 16` (programs/registry/src/lib.rs:46), so a given user always lands on the same shard but the 16 shards absorb concurrent registrations without a global write lock.

The corollary: **global accounts (config, totals) are read-only on hot paths and stale on purpose**. Periodic admin instructions (aggregate_readings in oracle, aggregate_shards in registry) fold per-entity state back into global totals.

4.3 Compute budget

Every instruction executes under a compute-unit meter (200k CU default per instruction, 1.4M per transaction ceiling). Exceeding it aborts the transaction. This repository profiles CU cost with the `compute_fn!` macro (shared/compute-debug/src/lib.rs:78), which logs remaining CU around each handler body on localnet and compiles to a no-op in release. Checkpoints inside long handlers use `compute_checkpoint!` (shared/compute-debug/src/lib.rs:143) — e.g. around the registry→energy-token CPI.

4.4 Zero-copy account access

State structs are `#[account(zero_copy)]` `#[repr(C)]` and accessed through `AccountLoader` (`load()` / `load_mut()` / `load_init()`). Instead of deserialising the whole account into heap objects (Borsh), the program casts the account's byte buffer in place — large accounts (sharded order books, meter state) incur low compute-unit cost. The cost is manual layout discipline: explicit padding fields, no `String` (fixed `[u8; N]` + length byte instead). See invariants 1–2 in [ARCHITECTURE.md](#).

5. Cross-Program Invocation (CPI)

Programs call each other synchronously inside one transaction; the callee inherits the transaction's account locks and compute budget. Two production CPI edges exist:

5.1 registry → energy-token (registration grant)

When a new user registers, the registry program mints the 10 GRX grant by CPI into energy-token. The registry signs the CPI **as its own PDA** using `CpiContext::new_with_signer` with the `[b"registry", bump]` seeds (programs/registry/src/lib.rs:298), then calls `energy_token::cpi::mint_tokens_direct` (programs/registry/src/lib.rs:305). This is the canonical PDA-signing pattern: no private key exists for the registry PDA; the runtime grants it signer status because the calling program proved the seeds derive to that address.

5.2 trading → governance (PoA config + ERC certificates)

Trading depends on governance with `features = ["cpi"]` and re-exports its types directly: `pub use governance:: {ErcCertificate, ErcStatus, GovernanceConfig}` (programs/trading/src/lib.rs:18). Settlement validates governance-owned accounts (deserialise + owner check) rather than invoking governance instructions — a read-side coupling, cheaper than a full CPI call.

Both edges are path dependencies in `Cargo.toml`, so Anchor builds callee CPI client code (typed `cpi:: modules`) at compile time — no runtime IDL lookup.

6. Security Model

Solana's runtime gives three primitives: **ownership** (only the owning program mutates an account), **signatures** (the runtime verifies tx signers before execution), and **PDAs** (addresses no private key can sign for, derivable only via the owning program). Everything else is program-level policy. This repository adds four mechanisms atop these primitives:

6.1 Anchor constraint checks (account validation)

Account structs declare constraints that Anchor verifies before the handler runs: `Signer<'info>` for required signers, `has_one = authority` for stored-authority matching, `seeds = [...]` + `bump` for PDA address verification. Governance gates every privileged instruction this way — e.g. `has_one = authority @ GovernanceError::UnauthorizedAuthority` on `IssueErc` (programs/governance/src/contexts.rs:31), `ValidateErc` (programs/governance/src/contexts.rs:90), `RevokeErc` (programs/governance/src/contexts.rs:108), and `UpdateGovernanceConfig` (programs/governance/src/contexts.rs:149).

6.2 Application-layer PoA with 2-step authority transfer

The governance program holds a GovernanceConfig account naming the platform authority. Authority rotation is two-phase to tolerate accidental key-entry errors:

1. `propose_authority_change` (programs/governance/src/handlers/authority.rs:13) — current authority writes `pending_authority` (authority.rs:34), guarded against an already-pending proposal (authority.rs:22).
2. `approve_authority_change` (programs/governance/src/handlers/authority.rs:53) — the **new** key must sign to accept, and only then does `poa_config.authority` flip (authority.rs:78).

An incorrectly entered new authority therefore cannot render the protocol inoperable — the incorrect key never signs the approval.

6.3 Off-chain-signed settlement: Ed25519 instruction introspection

The CDA matching engine runs off-chain (Trading Service). Matches settle on-chain via `settle_offchain.rs` without the buyer/seller being transaction signers. Instead:

- The off-chain engine collects **Ed25519 signatures from buyer and seller over the order payload** and places them in the transaction as instructions to Solana’s native Ed25519 verification program (which the validator executes — and rejects the transaction if invalid — before the trading instruction runs).
- The trading handler then **introspects the transaction** via the instructions sysvar: `verify_ed25519_signature(.../settle_offchain.rs:651)` calls `load_instruction_at_checked` (settle_offchain.rs:657), asserts the instruction targets the Ed25519 program (settle_offchain.rs:660), and asserts the pubkey embedded in the verified instruction matches the expected order owner (settle_offchain.rs:669). Buyer and seller are checked at instruction indexes 0 and 1 (settle_offchain.rs:310, settle_offchain.rs:314).

This is the standard Solana pattern for “verify an arbitrary off-chain signature on-chain” — the expensive curve math runs in the native program; the application program only proves the verification happened in the same transaction, against the right key and message.

Replay protection comes from OrderNullifier PDAs seeded per (user, order_id) (settle_offchain.rs:112). Each nullifier tracks `filled_amount`; a settlement may only consume the unfilled remainder (settle_offchain.rs:344) and increments the fill saturatingly (settle_offchain.rs:428), with the nullifier’s stored authority re-checked against the payload (settle_offchain.rs:539). The same signed order can therefore be partially filled across transactions but never over-filled or replayed.

6.4 REC-validator gating on mint

GRID tokens represent physical energy (1 kWh = 1 GRID), so minting is gated behind registered REC validators in energy-token. Validators are added/removed by the admin (programs/energy-token/src/lib.rs:280, lib.rs:313). The gate is **two-tier**. The attested-issuance paths — `mint_to_wallet` and the generation `mint_mint_generation` (§11.7) — require a REC co-signer **unconditionally**: the signing key must appear in the validator set (lib.rs:127–128, lib.rs:203) and an empty set rejects every such mint, with no opt-out. `mint_generation` tightens this to a **2-of-2**, additionally requiring the REC co-signer to differ from the program authority (lib.rs:146–149), so no single key can both authorise and attest a generation mint. Only the low-privilege registry-grant path `mint_tokens_direct` (the 10-GRX registration grant of §5.1) is **conditionally** gated — it enforces membership only once at least one validator is registered (`rec_validators_count > 0`, lib.rs:119, lib.rs:240) — so bootstrap registrations proceed before the REC set is populated while every energy-backed issuance stays gated.

6.5 Trust boundary summary

Boundary	Enforced by
Who may mutate an account	Runtime ownership rule (program-owned accounts)
Who may invoke privileged instructions	Anchor Signer + <code>has_one</code> against GovernanceConfig / stored authorities
Whether a trade was authorised	Ed25519 native-program verification + sysvar introspection
Whether a trade can be replayed	OrderNullifier PDA per (user, order)
Whether energy backing is real	REC validator set in energy-token
Who may even reach the RPC port	Off-chain: Chain Bridge mTLS + RBAC (superproject concern)

7. Protocol Flow (end-to-end runtime view)

A full market cycle touches all five programs:

1. Registration registry: create user PDA on shard (key[0] % 16)
└CPI→ energy-token: mint 10 GRX grant (PDA-signed)
2. Telemetry oracle: AMI gateway submits readings → per-meter MeterState PDA
(parallel across meters; 15-min market-clearing epochs)
3. Order entry trading: submit_order → per-order PDA on order shard
4. Matching OFF-CHAIN: Trading Service CDA engine matches buy/sell,
collects buyer+seller Ed25519 signatures
5. Settlement trading: settle_offchain_match – ed25519 ix at index 0/1,
introspection check, nullifier fill update, escrow transfer
(reads governance GovernanceConfig / ERC certificates)
6. Token movement energy-token / SPL: GRID + GRX transfers, REC-gated mint/settle
7. Reconciliation admin: aggregate_readings / aggregate_shards fold shard + meter
state into global totals (deliberately stale between runs)

Steps 2 and 3 are the throughput-critical paths and are exactly the ones designed for Sealevel parallelism (§4.2). Step 5 is the security-critical path (§6.3).

8. Consensus & Validator Topology

8.1 What Solana consensus provides

On any Solana cluster, ordering and finality come from **Proof of History** (a verifiable delay function giving each entry a position in time) feeding **Tower BFT** (a PoH-timestamped variant of practical BFT voting in which validators commit to forks with exponentially growing lockouts). Block production rotates across a leader schedule weighted by stake. The programs in this repository are consensus-agnostic: they see only the SVM account model and cannot tell which consensus produced the block.

8.2 What “permissioned PoA” means here

GridTokenX targets a **permissioned cluster**: the validator set is a closed list of known operators (utility / market-operator nodes) rather than open stake-weighted entry. Solana does not have a separate “PoA mode” — permissioning is operational (who is allowed to run a validator and receive stake delegation), and the **application-layer** PoA lives in the governance program’s GovernanceConfig (§6.2), which gates protocol administration regardless of who validates blocks. The two layers must be kept distinct:

Layer	Authority	Mechanism
Block production / finality	Permissioned validator operators	PoH + Tower BFT among allowlisted nodes
Protocol administration	GovernanceConfig.authority	Governance program checks (§6.1–6.2)
Energy attestation	REC validator set	energy-token gating (§6.4)

8.3 Development topologies

Mode	Nodes	Consensus reality	How
anchor test / localnet	1 × solana-test-validator	None — single node self-produces blocks; PoH runs but no voting quorum	anchor test, or superproject just solana-up
Surfpool simnet	0 local validators	Simulated against mainnet state	npm run simnet / npm run simnet:ci
Target deployment	N permissioned validators	Real Tower BFT among allowlisted operators	Out of scope for this repository

The single-node localnet means dev/test never exercises fork choice, leader rotation, or vote lockouts — only the SVM semantics (§4) are faithfully reproduced. Performance numbers from [BENCHMARKS.md](#) are therefore SVM/runtime measurements, not consensus-throughput measurements.

9. Key Results and Discussion

This section reports the principal empirical findings and interprets them against the design goals of §2.1. All figures are drawn from the canonical benchmark report [BENCHMARKS.md](#). The compute-unit (CU) values are machine-independent and are reproduced from the in-process LiteSVM profiles, whereas the throughput figures are qualified by the single-node caveat of §2.3.

Table 1 summarises the headline findings; each is developed and cited in the subsections that follow. Two throughput limits are reported separately and must not be conflated: the matching/OLTP path scales super-linearly to the swept ceiling, whereas any path that funnels through one write-locked account (mint issuance, single-fee-payer settlement, single-gateway meter ingest) is bounded by that serialisation, not by execution.

Metric	Measured value	Bottleneck source
Peak throughput (TPC-C proxy, c=40)	$\approx 74.25 \text{ tx} \cdot \text{s}^{-1}$	block-time + write-lock on shared accounts
Scaling (c=5 $\rightarrow$ c=40)	super-linear (9.4 $\times$ over 8 $\times$), no saturation knee	— (execution is load-independent)
Per-tx compute (all loads)	flat $\approx 22\text{--}24 \text{ k CU}$	load-independent (not execution-bound)
Single-signer mint issuance	$\approx 5.33 \text{ mint} \cdot \text{s}^{-1}$	shared mint write-lock (device-count independent)
Batch-settle (single fee-payer)	$\approx 0.6 \text{ tx} \cdot \text{s}^{-1}$	global collectors + treasury_state accumulator write-lock
Meter ingest, 100,000 meters	0.35% delivery loss, $\approx 13.5 \text{ k CU}$ flat	single gateway payer write-lock (not per-meter)
Transaction failure rate (TPC-C sweep)	0% at all concurrency levels	—
Settlement compute cost	121,813 CU·match ⁻¹ ($\approx 61\%$ budget)	Ed25519 verify + dual-mint escrow

Table 1: Headline empirical results. All values are drawn from the canonical benchmark report [BENCHMARKS.md](#) (3cb3388, Solana 3.1.10, Apple M2, 2026-07-06). CU figures are machine-independent; absolute throughput is qualified by the single-node caveat (§2.3, §8.3).

9.1 Compute-unit cost of on-chain operations

Table 2 summarises the measured CU cost of representative instructions and their share of the 200,000-CU default per-instruction budget. Two facts stand out. First, every control, telemetry, and settlement-recording path costs no more than approximately sixteen thousand CU — at most about eight per cent of the budget — so the high-frequency operations retain ample headroom. Second, the off-chain-match settlement instruction, `settle_offchain_match`, is by a wide margin the most expensive operation at roughly 122,000 CU (about 61% of the budget); its cost is dominated by the native Ed25519 signature verification and instruction-sysvar introspection that authorise the trade (§6.3), not by the settlement arithmetic itself.

Instruction	CU	% of 200k
<code>do_nothing</code> (dispatch + signature-verify floor)	769	0.4%
<code>treasury.record_settlement</code>	3,301	1.7%
<code>governance.approve_authority_change</code>	5,784	2.9%
<code>treasury.record_settlement_sharded</code>	5,374	2.7%
<code>oracle.trigger_market_clearing</code>	8,390	4.2%
<code>oracle.submit_meter_reading</code> (steady-state)	13,376	6.7%
<code>oracle.submit_meter_reading</code> (first, inits PDA)	16,050	8.0%
<code>trading.settle_offchain_match</code>	122,000	61.0%

Table 2: Measured compute-unit cost of representative instructions and their share of the 200,000-CU default budget. Source: [BENCHMARKS.md](#) §1, §4, §5, §5b.

The hot telemetry write, `submit_meter_reading`, costs about 16,050 CU on the first submission for a meter — which initialises the meter PDA — and about 13,376 CU in steady state, confirming that the dominant recurring on-chain write sits comfortably inside budget. The governance and oracle control instructions all fall between roughly 5,800 and 11,100 CU.

9.2 Parallelism and throughput under load

Under the TPC-C concurrency sweep, aggregate throughput rises from 7.87 transactions per second (TPS) at concurrency 5 to 74.25 TPS at concurrency 40 — a 9.4-fold increase over an 8-fold rise in concurrency — climbing monotonically with no saturation knee within the swept range. Crucially, the per-transaction CU cost remains flat at approximately 22,000–24,000 CU across all concurrency levels, which shows that on-chain execution cost is independent of load. The throughput ceiling is therefore imposed by consensus block time and by write-lock serialisation on the few genuinely shared accounts, not by program execution.

The single-node development topology bounds absolute write throughput at approximately 2.0 TPS for any operation that requires a block, because latency is dominated by the 400–600 ms block time rather than by compute. By contrast, a read served directly by the RPC node returns in under a millisecond ($\approx 1,454$ TPS), three orders of magnitude faster, as it involves no consensus round-trip. These absolute rates are properties of the single-node harness, not of the protocol, and must not be read as consensus-throughput results (§2.3, §8.3).

A distinct serialisation limit governs token issuance. Settlement mints GRID against a matched trade, and because every settle write-locks the same treasury accumulator and the shared fee, wheeling, and loss collector accounts, issuance does not parallelise across the contributing devices: an open-loop sweep in which a single fee-payer submits settlements holds at approximately $0.6 \text{ mint}\cdot\text{s}^{-1}$ irrespective of in-flight concurrency and of whether the offered load originates from one device or many. The bottleneck is the shared accumulator, not the signer or the device population — the direct-mint path, by contrast, packs many mints into a single slot from one signer, confirming that raw issuance is limited by the confirmation round-trip rather than by on-chain contention. Two complementary remedies restore parallelism: sharding the collector and accumulator accounts so that concurrent settlements touch disjoint writable state (§9.5), and pooling multiple fee-payers so that submission is not funnelled through a single account. In a Tier-A prototype combining both, per-slot settlement packing rises to approximately $7.5 \text{ mint}\cdot\text{s}^{-1}$. We therefore identify multi-signer fee-payer pooling, together with per-shard collector accounts, as the primary lever for settlement-side throughput.

Table 3 consolidates every throughput figure measured in this evaluation. All values are **client-observed confirmed goodput** — successfully confirmed transactions per wall-clock second, measured from burst start to last observed confirmation at confirmed commitment, so each figure includes the full pipeline (client signing, RPC ingest, banking-stage execution, block production, and confirmation visibility). None is a consensus- or network-throughput result (§2.3, §8.3); rows measured through different client harnesses (the OLTP proxy’s per-transaction confirmation versus the ingest benchmark’s raw submission) are not directly comparable with one another.

Path	TPS	Bottleneck / regime
RPC read (no consensus round-trip)	$\approx 1,454$	none — RPC node local
Meter ingest, steady (5k–50k meters, sustained)	393–490	single gateway fee-payer write-lock
Meter ingest, steady (100k meters)	322	single gateway fee-payer write-lock
CDA order entry, sharded (10k orders)	180	shared zone_market write-lock + fee-payer
TPC-C OLTP proxy (concurrency 5 $\rightarrow$ 40)	7.87 $\rightarrow$ 74.25	block time + shared-account locks
Sequential single-operation write	≈ 2.0	400–600 ms block time per round-trip
Settlement mint, Tier-A prototype (sharded + pooled payers)	≈ 7.5	per-slot packing
Settlement mint, single fee-payer	≈ 0.6	global collector/accumulator write-lock

Table 3: Consolidated measured throughput. All values are client-observed confirmed goodput on the single-node validator; harnesses differ per row and rows are not mutually comparable. Source: [BENCHMARKS.md](#) §3, §9 and the settlement sweeps of §9.2.

The spread spans three orders of magnitude and sorts cleanly by how much shared, write-locked state each path touches: reads lock nothing; meter ingest locks only the gateway payer; order entry additionally write-locks the shared zone-market account; the OLTP proxy locks a few market accounts; and naive settlement locks global accumulators. Order entry makes the ordering principle explicit: it is the cheapest hot-path write measured ($\approx 9.9 \text{ k CU}$, below the $\approx 13.5 \text{ k CU}$

meter write) yet delivers $2.6\times$ lower throughput than meter ingest at the same fleet size, because its account contexts still declare the shared zone-market account writable although the handler mutates only the per-shard account. Throughput on this platform is a function of lock footprint, not of instruction compute cost. The formal workload definition and the full order-entry measurement are given in §11.11.

9.3 Meter-telemetry ingest scaling to 100,000 meters

The concurrency sweep above uses a generic-OLTP proxy. To characterise the **actual** telemetry hot path at fleet scale, we constructed a fleet-ingest benchmark (`scripts/bench-meter-throughput.ts`) that emulates an AMI gateway submitting one reading per meter for fleets of 80 up to 100,000 distinct meters against a live single-node validator.

Test design. Each simulated meter has a unique identifier, so each submission targets its own MeterState PDA (`[b"meter", meter_id]`) and the write set of any two submissions is disjoint (Sealevel-parallelisable, §4.2). All submissions are signed by one gateway key — the registered `chain_bridge` authority that the oracle program requires (§7 step 2) — which mirrors the production topology in which a single Aggregator Bridge fronts the meter fleet, and which deliberately preserves the shared-fee-payer serialisation the paper identifies as the ingest ceiling. Every run submits **two epochs** of readings: the on-chain rate-limit guard (`min_reading_interval = 60 s`) and the strictly-increasing-timestamp guard force epochs at least 61 s apart, and the split separates the one-off account-creation cost (epoch 1 initialises every meter PDA and pays its rent) from the recurring steady-state write (epoch 2), which is the figure that matters for a fleet in operation. Reading values follow a fixed synthetic pattern; each transaction carries exactly one `submit_meter_reading` instruction.

Measurement. Throughput is client-observed confirmed goodput as in Table 3; latency is send-to-confirmed per transaction; compute units are read post-hoc from transaction metadata of a sample of confirmed transactions. Every non-confirmed transaction is attributed to one of three failure classes: **send-rejected** (the RPC node refused the submission — never entered the validator, no fee), **on-chain error** (executed and failed a program guard — fee paid, failure recorded on the ledger), and **expired** (accepted but never confirmed within 90 s — dropped from the ingest queue or its blockhash aged out; no fee, no trace). The classes have different operational meaning: on-chain errors are the oracle’s input validation doing its job, while send-rejected and expired transactions are pure delivery loss, invisible on-chain and recoverable by client retry — the rate-limit and monotonic-timestamp guards make resubmission idempotent: no reading can be double-counted. All fleet sizes use one transport: transactions are pre-signed locally against a cached blockhash, submitted raw without preflight or retry, and confirmed by bulk signature-status polling (1.5 s sweep) under a 3,000-transaction in-flight window, so latency is poll-quantised (± 1.5 s). Reading timestamps are taken from the on-chain clock rather than the client’s wall clock, matching the clock against which the oracle’s freshness guard is evaluated. The formal metric definitions and the full harness parameterisation are given in §11.10. Table 4 reports the result.

Meters	Confirmed / total	Validation- rejected	Delivery loss	Steady CU min	TPS (steady)
80	160 / 160	0%	0%	13,560	47
1,000	1,988 / 2,000	0.40%	0.20%	13,634	267
5,000	9,906 / 10,000	0.48%	0.46%	13,634	393
10,000	19,862 / 20,000	0.46%	0.23%	13,337	462
50,000	99,247 / 100,000	0.47%	0.28%	13,337	490
100,000	198,347 / 200,000	0.48%	0.35%	13,634	322

Table 4: Oracle meter-telemetry ingest scaled from 80 to 100,000 meters on a live single-node validator, with every non-confirmed transaction attributed. Validation-rejected = the oracle’s anomaly gate firing on the synthetic value pattern (§11.10); delivery loss = send-rejected + expired, with no client retry. Steady-state CU is the recurring per-meter write; the base path (CU min) is the value to cite. Source: [BENCHMARKS.md](#) §9.

Three results stand out. First, the steady-state base write cost holds at 13.3–13.6 thousand CU across a 1,250-fold increase in meter count, matching the 13,376 CU litesvm figure of §9.1: the per-meter write is $O(1)$ in fleet size, exactly as the per-entity-PDA partitioning predicts, with the residual ± 150 CU attributable to meter-identifier byte length rather than fleet size. Second, the loss decomposition separates two very different phenomena. The validation-rejected column is constant at $\approx 0.47\%$ because the synthetic value pattern deterministically drives that fraction of meters past the anomaly gate (Equation 17) — the same meters are rejected in every epoch, each rejection is fee-paid and recorded, and the bucket demonstrates the oracle’s input validation firing correctly under 100,000-meter load. True delivery loss — submissions that vanished in transit — stays at or below 0.46% at every fleet size (0.35% at 200,000 transactions) with no client retry; because resubmission is idempotent, a single retry round would reduce the effective rate to the order of 10^{-5} .

Third, throughput does not grow with meter count: small fleets are ramp-limited (an 80-transaction burst never fills the pipeline), sustained rates reach roughly 390–490 TPS between 5,000 and 50,000 meters, and the fleet doubling from 50,000 to 100,000 does not raise it, because every submission is signed by the single gateway fee-payer, which is write-locked on each transaction — as with settlement in §9.2, the ceiling is a shared-account serialisation limit, not per-meter contention. The flat CU confirms the on-chain write itself is already scale-free; converting the disjoint per-meter PDAs into proportional throughput requires the same remedy identified for settlement, namely pooling multiple gateway fee-payers so ingress is not funnelled through one signer.

9.4 One-month community simulation: closing the token lifecycle

The preceding sections measure each hot path in isolation. To demonstrate that the paths compose — and that the token accounting closes — we replayed one full month of a physically modelled community against a live validator and drove every stage of the token lifecycle from the resulting data (`scripts/bench-community-month.ts`). The input is not synthetic in the sense of §9.3: it is produced by the platform’s smart-meter simulator (`gridtokenx-smartmeter-simulator`, seeded solar-irradiance and load models), which generates an 80-meter fleet containing 12 solar prosumers and 68 consumers, sampled at the production cadence of 96 fifteen-minute intervals per day for 30 days — 230,400 readings totalling 15,868.5 kWh generated, 144,879.8 kWh consumed, and 10,386.7 kWh of interval surplus. A market-policy constraint caps each prosumer at 10 kWh of sales per day; the cap binds on 305 of 360 prosumer-days.

Phase design. The oracle rejects only future-dated readings, so a month of historical timestamps replays back-to-back. Because the per-meter strictly-increasing-timestamp guard forbids two in-flight submissions for one meter, readings ship as per-meter **batched** transactions — up to ten `submit_meter_reading` instructions per transaction, whose in-transaction sequential execution preserves order — across 80 concurrently progressing meter chains; readings that would trip the anomaly gate (Equation 17) are isolated into single-instruction transactions so the on-chain rejection is still exercised rather than aborting a batch. Each simulated day then runs a trading session (12 sell offers of $\min(\text{day surplus}, 10 \text{ kWh})$, 68 bids at actual daily consumption) and the full lifecycle per prosumer: a registry synchronisation plus `settle_and_mint_tokens` CPI mints the capped, **oracle-accepted** surplus as GRID (Token-2022); the seller deposits it into escrow; an Ed25519-signed off-chain match settles against a rotating consumer through `settle_offchain_match` (§6.3) in a v0 transaction with an address-lookup table; the buyer withdraws delivery and burns it (energy consumed = tokens retired). At month end the cap-withheld surplus is certified as renewable-energy certificates through `governance_issue_erc`, whose `mark_erc_claimed` CPI enforces on-chain that GRID plus REC claims never exceed metered generation (§11.7). Table 5 reports the lifecycle stages of the canonical run.

Stage	ok	fail	CU median	latency p50 (ms)
registry sync + GRID mint (CPI)	433	0	29,410	1,468
escrow deposit	433	0	14,905	1,480
off-chain-signed settlement (Ed25519 ×2, v0+ALT)	353	0	107,141	1,465
withdraw + burn	353	0	21,963	1,475
REC issuance (<code>issue_erc</code>)	12	0	71,369	1,471

Table 5: Token-lifecycle stages over the simulated month (canonical run, 2026-07-07): 353 prosumer-days cleared the 0.1 kWh dust floor and completed the full mint→deposit→settle→burn chain; 12 month-end REC issuances. No stage recorded a failure. Latency is send→confirmed, poll-quantised (± 1.5 s). Source: [BENCHMARKS.md](#) §11.

Results. The month was absorbed in 1,394.7 s of wall time — a 1,858-fold real-time compression — with zero delivery loss on the telemetry phase: the 919 missing readings are exactly the anomaly-gate rejections, fee-paid and recorded on the ledger, and this outcome reproduced **bit-identically across seven replays** (throughput varied 194–232 readings/s with host load; the batched transport carries ≈ 213 readings/s at only ≈ 21 tx/s). The token accounting closes exactly: 3,290.724 kWh of GRID was minted, all of it trade-settled and subsequently burned, and 4,962.778 kWh was certified as RECs — the two sums together equal the oracle-accepted interval surplus of 8,253.502 kWh to the watt-hour. The sell cap withholds 68% of raw surplus from the market, and total demand (126,983.8 kWh) exceeds capped supply (3,290.8 kWh) thirty-nine-fold, quantifying the cap’s economic bite in this community.

On-chain verification. Every headline figure was re-derived from live chain state through RPC reads alone (`scripts/audit-community-month.ts`, 15/15 assertions): the per-meter reading counters sum to 229,481; the order, trade-nullifier, and order-nullifier account counts are 2,396, 353, and 706; the registry’s `settled_net_generation` and `claimed_erc_generation` sums equal the minted and certified totals; the GRID supply after burns collapses to the 68 escrow-seed watt-hours, proving every traded token was retired; the REC supply equals the claimed energy at the mint’s

1,000-base-units-per-kWh scale; and the settlement currency conserves exactly (buyer escrow outflow 13,137 minor units = seller proceeds 12,832 + wheeling charges 305).

A reproducibility caution. An early lifecycle run silently lost 112 of 353 settlements to transaction **deduplication**: consecutive cap-bound days produced byte-identical escrow-deposit transactions (same instruction, accounts, fee payer, and cached blockhash within its 10 s refresh window), so the validator deduplicated the resend while the client observed the earlier signature as confirmed and proceeded — leaving the escrow unfunded at settlement. Replay harnesses that reuse cached blockhashes must salt every transaction (ours now prepends a day-varying compute-budget instruction); we record this because the failure mode is invisible to the sender and surfaces only downstream.

9.5 Discussion

The measurements support the central design claim of §2.1: partitioning high-frequency state into per-entity PDAs removes execution-side contention, so the residual scaling limit is consensus and lock serialisation rather than compute. The flat CU-versus-concurrency profile is the direct evidence — had hot paths shared a writable account, compute cost or failure rate would have risen with load; neither does. The sharded settlement-recording path reinforces this: `record_settlement_sharded` writes a per-shard PDA at about 5,374 CU instead of contending on a single global counter, converting a would-be serialisation point into an embarrassingly parallel one, at the cost of a periodic off-hot-path aggregation.

Settlement is the natural cost centre, and the 122,000-CU figure quantifies the price of trustless authorisation: verifying two counterparty signatures on-chain and proving that verification through `sysvar` introspection. This remains a single transaction well within the per-transaction budget, but it explains why batch settlement processes one match per transaction — the Ed25519 signature data for each match cannot be compressed through an address-lookup table — so batching amortises account-setup overhead rather than signature cost. Reducing this figure, for example through succinct proof aggregation over many matches, is the clearest avenue for further compute savings.

The principal limitation of this evaluation is topological. Because all on-chain measurements are taken on a single self-producing validator, they faithfully characterise SVM semantics — account locking, compute metering, and per-instruction cost — but say nothing about consensus throughput, fork choice, or leader rotation, which would require a multi-validator permissioned deployment (§8.3). Establishing sustained end-to-end throughput on such a deployment, and locating the true saturation point above concurrency 40, are left to future work.

9.6 Settlement under the three price rules

To confirm that the choice of price rule (§11.2) is a purely economic degree of freedom with no on-chain compute cost, we settled real matches through `batch_settle_offchain_match` on the live single-node validator under each rule across two fleet sizes. Each transaction settles one match (the single-match packet cap of \$9.5); the off-chain match sets the clearing price p^* to the rule’s value inside the signed band $p_s = 2.00 \leq p^* \leq p_b = 2.10$ THBG/kWh, and N is the number of independent matches submitted at concurrency 4. Table 6 reports confirmed goodput and the per-settle compute cost read from transaction metadata.

Price rule	p^* (¢/kWh)	N	settled	TPS	CU/settle
CDA on-chain ($p^* = p_s$)	2.00	4	4/4	0.21	116,470
CDA on-chain ($p^* = p_s$)	2.00	8	8/8	0.45	114,408
Uniform-price epoch	2.04	4	4/4	0.23	116,470
Off-chain midpoint	2.05	8	8/8	0.46	115,533

Table 6: Live on-chain settlement under each price rule and fleet size (N matches, concurrency 4) on the single-node validator (Agave 3.1.10, Apple M2). Each settle is one `batch_settle_offchain_match` transaction; CU is machine-independent (computeUnitsConsumed), TPS is client-observed confirmed goodput (§9.2). Seller-net/buyer-cost per rule are the deterministic values of Table 9 and are not re-measured on-chain.

Three observations. First, the per-settle compute cost holds at 114–116 k CU across all three price rules — a spread below 2% that tracks incidental checked-arithmetic branches on the differing currency magnitudes, not the rule itself — confirming directly on-chain that price selection does not touch settlement compute; the figure also agrees with the 121,813 CU litesvm profile of §9.1 to within the localnet-vs-litesvm margin. Second, every match settled (100%, zero on-chain reverts) at every rule and fleet size, so the price rule is economically free: it redistributes the spread (Table 8) and the take-home currency (Table 9) without changing whether or how the trade lands. Third, confirmed goodput rises with the offered match count — ≈ 0.22 TPS at $N = 4$ to ≈ 0.45 TPS at $N = 8$ — while remaining in the sub-1-TPS, single-

fee-payer regime of §9.2, since all matches in a run share one settlement fee-payer and the global collector/accumulator write-locks that §9.2 identifies as the settlement ceiling. The measurement thus reproduces, on the real settlement path, both the price-invariance of compute and the shared-account throughput bound established for the OLTP proxy.

To trace the throughput of the settlement path directly, we then swept the fleet from 80 to 720 prosumer meters (each match pairs two meters, so $N = 40\text{--}360$ matches) at the CDA rule, holding concurrency at 4, and settled every match on the live validator. We report on-chain throughput as **slot density** – settles landed per slot times the slot rate (≈ 2.5 slot·s⁻¹ at the ≈ 400 ms slot time) – because it is read from the confirmed transactions’ slot numbers and is therefore immune to the client-side confirmation-poll latency that inflates a naive wall-clock rate. Table 7 reports it alongside the per-settle compute.

Meters	Matches N	settled	settles/slot	on-chain TPS	CU/settle
80	40	40/40	1.21	3.03	115,008
160	80	80/80	2.05	5.13	117,258
360	180	180/180	5.29	13.24	115,053
720	360	360/360	10.91	27.27	116,366

Table 7: Live settlement throughput versus fleet size (CDA rule, concurrency 4, single-node validator). Each match pairs two meters and is one settle transaction. Slot density = settles landed per slot; on-chain TPS = slot density $\times$ 2.5 slot·s⁻¹.

Every settle confirmed (0 reverts) at every size; CU is machine-independent.

On-chain throughput scales **linearly** with fleet across the full range: the settles-per-slot figure rises in near-exact proportion to the match count (≈ 0.03 settles·slot⁻¹ per match at every size), so the burst occupies a near-constant span of $\approx 33\text{--}39$ slots regardless of N and the validator simply packs proportionally more settles into each slot – from 1.2 at 80 meters to 10.9 at 720. On-chain throughput therefore climbs from 3.0 to 27.3 settles·s⁻¹ over a nine-fold fleet increase, with every one of the 660 settlements across the sweep confirming and none reverting. The shared collector and treasury_state accumulator that every settle write-locks (§9.2) serialise **execution within** a slot but do not stop the banking stage from co-locating many conflicting settles in the same slot, so no saturation knee appears in the tested range – the 720-meter fleet lands at 10.9 settles per slot with 100% success. Compute is flat throughout, 115.0–117.3 k CU across all four sizes – the same ≈ 115 k CU that is invariant to both price rule and fleet size – reconfirming that settlement cost is fixed and the throughput story is entirely one of slot packing, not execution. (The client-side wall-clock rate is lower and noisier – 2.3–22.9 TPS over the same runs – because it folds in confirmation-poll latency; the slot-density figure is the machine-level throughput and the one reported here.)

10. Reading Map

Question	Read
What are the programs / IDs / invariants?	ARCHITECTURE.md
Zero-copy layout, sharding, CU profiling detail	SKILL.md (versions/IDs stale – trust Anchor.toml)
Benchmark methodology + results	BENCHMARKS.md
Settlement math / clearing	programs/trading/src/instructions/settle_offchain.rs
Platform-wide rules (Chain Bridge, gateways)	superproject ../CLAUDE.md, ../ARCHITECTURE.md

11. Formal Model: Pricing, Settlement & Economic Equations

All on-chain money math uses integer checked_mul with u128 intermediates – overflow **rejects** the transaction, never clamps. $\lfloor \cdot \rfloor$ denotes the integer floor division the SBF program performs.

11.1 Notation

Symbol	Meaning	Scale
q	matched energy quantity (match_amount)	9-dec, 1 kWh = 10^9
p	clearing price (match_price)	6-dec THBG/kWh
V	trade gross value	6-dec THBG
φ_m	market fee (market_fee_bps)	bps

Symbol	Meaning	Scale
w	wheeling rate (wheeling_rate_per_kwh)	6-dec THBG/kWh
ℓ	loss rate (loss_bps)	bps
r	peg rate (grx_per_thbg_rate)	THBG-minor / whole GRX
φ_s	swap fee (swap_fee_bps)	bps
A	staking accumulator (acc_reward_per_share)	$\times 10^{12}$
T	total staked GRX (total_staked)	atoms

11.2 Price formation

Prices are limit prices quoted in THBG per kilowatt-hour (6-decimal fixed point). The market operates two coordinated mechanisms — a continuous double auction for immediate execution and a uniform-price auction over 15-minute epochs — and both resolve to the same settlement arithmetic of §11.3.

Order admission. Every order carries a strictly positive limit price and must fall within the market’s configured band (programs/trading/src/lib.rs:221, lib.rs:226–233); orders outside $[p_{\min}, p_{\max}]$ are rejected at entry, which bounds the price domain before any matching occurs:

$$p_{\min} \leq p \leq p_{\max} \quad (1)$$

Continuous double auction (on-chain path). Two orders may match only when they cross (programs/trading/src/lib.rs:454):

$$p_b \geq p_s \quad (2)$$

and the execution price is the seller’s limit (lib.rs:462; instructions/sharded_match_orders.rs:40):

$$p^* = p_s \quad (3)$$

i.e. the full bid–ask spread accrues to the buyer, a deliberately conservative rule for a market in which buyers are predominantly consumers.

Off-chain matching (settled on-chain). The off-chain CDA engine is free to choose any execution price, but settlement enforces that the price lies within the limits **signed by both counterparties** in their Ed25519 payloads (instructions/settle_offchain.rs:718–719, SlippageExceeded):

$$p_s \leq p^* \leq p_b \quad (4)$$

so no participant can be settled at a price worse than the limit they signed, regardless of engine behaviour. The reference engine splits the spread at the midpoint, $p^* = \lfloor (p_b + p_s)/2 \rfloor$ (scripts/simulate-market-clearing.ts:131), which divides the surplus equally between the counterparties.

Uniform-price epoch clearing. Accumulated bids clear at a single price per 15-minute epoch. The oracle admits a clearing trigger only on an exact 900-second boundary strictly newer than the last cleared epoch (programs/oracle/src/lib.rs:202–212), and each executed match records its price as the zone’s last_clearing_price (programs/trading/src/lib.rs:494; settle_offchain.rs:946).

Price transparency. The market maintains a 24-slot ring buffer of recent trade prices and recomputes a volume-weighted average price on each update (programs/trading/src/lib.rs:820–863):

$$\text{VWAP} = \left\lfloor \frac{\sum_i p_i v_i}{\sum_i v_i} \right\rfloor \quad (5)$$

Network charges (market fee, wheeling, and loss, §11.3) are levied on top of the execution price and are bounded by the 20% cap of Equation 10, so the effective delivered price a buyer faces is p^* plus a capped, auditable surcharge.

Surplus allocation across price rules. The three market rules above — and a non-market feed-in baseline — divide the same bid–ask spread $\Delta = p_b - p_s$ differently, while the settlement arithmetic of §11.3 applies identically on top of whichever p^* each rule selects. Writing the seller’s spread share as $(p^* - p_s)/\Delta$ and the buyer’s as its complement $(p_b - p^*)/\Delta$, the rules span the full range from buyer-favourable to even-split, and a feed-in tariff removes the bilateral trade entirely — paying the seller an exogenous rate p_{fit} regardless of any bid, so the surplus $q\Delta$ accrues to the single

off-taker rather than the counterparties. Table 8 states each rule and works a representative intra-zone match at $p_s = 3.00$, $p_b = 4.00$ THBG/kWh ($\Delta = 1.00$), with an illustrative $p_{\text{fit}} = 2.20$ THBG/kWh – a reference export rate, not a measured value.

Price rule	p^*	Seller	Buyer	Source
CDA on-chain (immediate)	$p_s = 3.00$	0%	100%	lib.rs:462; sharded_match_orders.rs:40
Off-chain midpoint (ref. engine)	3.50	50%	50%	simulate-market-clearing.ts:131
Uniform-price epoch ($p_c = 3.40$)	3.40	40%	60%	lib.rs:494; oracle lib.rs:202–212
Feed-in tariff (non-market)	$p_{\text{fit}} = 2.20$	—	—	exogenous off-taker; no Equation 4 match

Table 8: Execution price and bid–ask-spread allocation under each price rule, for a representative match ($p_s = 3.00$, $p_b = 4.00$ THBG/kWh, $\Delta = 1.00$). Network charges (§11.3) apply identically on top of p^* and are omitted here to isolate the spread split. All three market rules keep p^* inside the counterparties’ signed limits (Equation 4); the feed-in row is a non-market baseline (no bilateral match) and p_{fit} is illustrative.

Relative to the 2.20 THBG/kWh feed-in baseline, even the most buyer-favourable market rule – CDA on-chain, $p^* = p_s$ – already lifts the seller to 3.00 THBG/kWh (a 36% premium) before charges, and the midpoint rule to 3.50 (+59%); the surplus a feed-in tariff would transfer wholesale to the single off-taker is instead divided between the two P2P counterparties. Because every market rule constrains p^* to the signed interval $[p_s, p_b]$ (Equation 4), the choice of rule only **redistributes** the fixed gains-from-trade $q\Delta$ between buyer and seller; it never settles either side outside the price they authorised, and a head-to-head welfare measurement of the three rules on matched trade data is left to future work.

11.3 Trade settlement

Gross value – rescale the 10^{15} -scaled product (9-dec energy $\times$ 6-dec price) back to 6-dec currency by the energy divisor $D_E = 10^9$ (settle_offchain.rs:1145):

$$V = \left\lfloor \frac{q \cdot p}{10^9} \right\rfloor \quad (6)$$

Market fee (settle_offchain.rs:788):

$$F_m = \left\lfloor \frac{V \cdot \varphi_m}{10^4} \right\rfloor \quad (7)$$

Wheeling – flat per-kWh, **not** ad-valorem; same energy rescale as Equation 6 (settle_offchain.rs:1160):

$$C_w = \left\lfloor \frac{q \cdot w}{10^9} \right\rfloor \quad (8)$$

Line loss (settle_offchain.rs:1189):

$$C_\ell = \left\lfloor \frac{V \cdot \ell}{10^4} \right\rfloor \quad (9)$$

Network charge and its cap invariant, $\Theta = 2000$ bps (20%); ChargesExceedCap if violated (settle_offchain.rs:116):

$$C_{\text{net}} = C_w + C_\ell, \quad C_{\text{net}} \leq \left\lfloor \frac{V \cdot \Theta}{10^4} \right\rfloor \quad (10)$$

Seller net proceeds, with feasibility $F_m + C_{\text{net}} \leq V$ enforced (ChargesExceedValue, settle_offchain.rs:123):

$$N_{\text{seller}} = V - F_m - C_{\text{net}} \quad (11)$$

Net proceeds under each price rule. Applying Equation 6–Equation 11 to the four price rules of Table 8 makes the take-home difference concrete. The buyer’s escrow debit is the gross value $V = qp^*$ and splits into seller proceeds plus the fee/wheeling/loss collectors (the on-chain conservation identity of §11.12), so the buyer’s delivered cost per kWh is p^* while the network charges reduce the **seller’s** net. Table 9 evaluates a $q = 10$ kWh intra-zone match under representative charges – market fee $\varphi_m = 25$ bps (the on-chain default, initialize_market.rs:23), wheeling $w = 0$ (intra-zone), and loss $\ell = 100$ bps – so the combined deduction is a flat 1.25% of V .

Price rule	p^* (£/kWh)	V (£)	fee+loss (£)	seller net (£/kWh)	vs feed-in
CDA on-chain ($p^* = p_s$)	3.00	30.00	0.375	2.9625	+34.7%
Uniform-price epoch (p_c)	3.40	34.00	0.425	3.3575	+52.6%
Off-chain midpoint	3.50	35.00	0.4375	3.4563	+57.1%
Feed-in tariff (baseline)	2.20	—	—	2.2000	—

Table 9: Seller net proceeds and buyer cost under each price rule, for a $q = 10$ kWh intra-zone match with $\varphi_m = 25$ bps, $w = 0$, $\ell = 100$ bps (deduction = 1.25% of V ; Equation 6–Equation 11). Buyer cost per kWh is p^* (the charges are carved out of the seller side, §11.12). The feed-in row is a non-market baseline: the seller is paid p_{fit} by a single off-taker with no P2P charges. Premium is seller net per kWh over the feed-in rate.

Two points follow. First, because the deduction is a fixed fraction of V and V is monotone in p^* , the ranking of seller net across rules is identical to the ranking of p^* itself: the price rule chosen sets the seller’s take-home, and the charges scale it uniformly rather than reordering it. Second, every market rule clears the feed-in baseline by a wide margin — the most buyer-favourable rule (CDA on-chain) still nets the seller 2.96 £/kWh after charges, a 34.7% premium over the 2.20 £/kWh export rate, and the midpoint rule 57.1% — so the on-chain charge load (1.25% here, capped at 20% by Equation 10) is small against the gains-from-trade the market unlocks relative to selling to a single off-taker.

11.4 Treasury peg (GRX $\leftrightarrow$ THBG)

Swap GRX $\rightarrow$ THBG, divisor $D_G = 10^9$ atoms/whole-GRX (treasury/src/lib.rs:58):

$$G = \left\lfloor \frac{g_{\text{in}} \cdot r}{10^9} \right\rfloor, \quad F = \left\lfloor \frac{G \cdot \varphi_s}{10^4} \right\rfloor, \quad n = G - F \quad (12)$$

Peg invariants — reserve attestation freshness and full backing (PegBreach, treasury/src/lib.rs:70):

$$t_{\text{now}} - t_{\text{att}} \leq \tau_{\text{ttl}}, \quad S_{\text{thbg}} + n \leq R_{\text{att}} \quad (13)$$

Redeem THBG $\rightarrow$ GRX with collateral guards $a_{\text{in}} \leq S_{\text{thbg}}$ (SupplyUnderflow) and $g_{\text{out}} \leq V_{\text{swap}}$ (InsufficientVault) (treasury/src/lib.rs:91):

$$g_{\text{out}} = \left\lfloor \frac{a_{\text{in}} \cdot 10^9}{r} \right\rfloor \quad (14)$$

11.5 MasterChef staking (GRX yield)

Accumulator update on fund_rewards, precision $\Pi = 10^{12}$, requires $T > 0$ (treasury/src/lib.rs:975):

$$A \leftarrow A + \left\lfloor \frac{a \cdot 10^{12}}{T} \right\rfloor \quad (15)$$

Pending reward and debt baseline for a position of size s ; Π absorbs the division loss (exactness unit-tested, lib.rs:126):

$$\rho = \max\left(0, \left\lfloor \frac{s \cdot A}{10^{12}} \right\rfloor - d\right), \quad d = \left\lfloor \frac{s \cdot A}{10^{12}} \right\rfloor \quad (16)$$

11.6 Oracle validation (15-min epoch)

Anomaly gate — integer cross-multiplication avoids float division, $\kappa = \text{max_production_consumption_ratio}$ (oracle/src/lib.rs:462):

$$E_{\text{prod}} \cdot 100 \leq \kappa \cdot E_{\text{cons}} \quad (17)$$

Reading quality score (oracle/src/lib.rs:370):

$$Q = \min\left(100, \left\lfloor \frac{100 \cdot n_{\text{valid}}}{n_{\text{valid}} + n_{\text{reject}}} \right\rfloor\right) \quad (18)$$

Range admission: $E_{\text{min}} \leq E \leq E_{\text{max}}$.

11.7 Surplus tokenisation (generation mint)

Surplus energy that survives the validation of §11.6 is tokenised by the energy-token program’s `mint_generation` instruction (§6.4): the off-chain pipeline aggregates a meter’s net surplus over a 15-minute window and the bridge submits one mint per (meter, window). The kWh-to-atoms conversion is a single function shared by the producer and the bridge verifier, so both sides agree on the exact bound and scaling (`gridtokenx-blockchain-core/src/rpc/nats_schema.rs:127`). A window surplus of E kWh (an IEEE-754 double on the wire) mints

$$A = \lfloor E \cdot 10^9 \rfloor, \quad \text{admitted iff } E \text{ is finite} \wedge 0 < E \leq E_{\max} \wedge A \geq 1 \quad (19)$$

with $E_{\max} = 10^6$ kWh per meter-window (`nats_schema.rs:117`). The cap is load-bearing rather than cosmetic: it bounds $A \lesssim 10^{15} \ll 2^{64}$ **before** the float-to-integer cast, which for an adversarially large f64 would otherwise saturate to $2^{64} - 1$ (Rust float casts saturate, not wrap) and request an astronomical mint. Non-finite, non-positive, over-cap, and rounds-to-zero values are rejected on both sides of the trust boundary.

On-chain, the mint executes only if three predicates hold (`programs/energy-token/src/lib.rs:278`, `lib.rs:287`, `lib.rs:308`):

$$w \equiv 0 \pmod{900 \cdot 10^3} \wedge w > 0, \quad M(m, w) = 0, \quad k_{\text{rec}} \in \mathcal{V} \quad (20)$$

The window start w (milliseconds) must sit on the same 900-second epoch grid the oracle’s clearing trigger enforces in seconds (§11.6); $M(m, w)$ is a per-(meter, window) idempotency record – a replayed mint intent is a no-op success, and the record is stamped only **after** a successful mint CPI, so a failed mint leaves the window retryable while a completed one can never double-mint; and the co-signing key k_{rec} must be a member of the registered REC-validator set $\mathcal{V}$, with no opt-out – an empty set rejects every generation mint (§6.4). The token supply then advances by exactly the attested amount, $S \leftarrow S + A$, tying issuance one-to-one to metered surplus at $1 \text{ kWh} = 10^9$ atoms.

11.8 Registry sharding & slashing

Deterministic shard from the first byte of the authority key k (`registry/src/lib.rs:71`):

$$\sigma(k) = k[0] \bmod 16 \quad (21)$$

Validator-active predicate, $\beta_{\min} = 10^{13}$ atoms = $10\{, \}000$ GRX (`registry/src/lib.rs:35`):

$$\text{active} \iff s_{\text{grx}} \geq \beta_{\min} \quad (22)$$

11.9 Parameters (data)

Parameter	Value	Meaning
D_E, D_G	10^9	energy / GRX atomic divisor
Π	10^{12}	staking accumulator precision
Θ	2000 bps	max network-charge cap (20%)
$\beta_{\min}$	10^{13} atoms	min validator stake (10k GRX)
shards	16	registry counter shards
epoch	900 s	oracle market-clearing window
$E_{\max}$	10^6 kWh	max single generation mint (Equation 19)
GRID/GRX dec	9	$1 \text{ kWh} = 10^9$ atoms
THBG dec	6	$1 \text{ THB} = 10^6$ minor
REC dec	6	$1 \text{ token} = 1 \text{ MWh}$

11.10 Fleet-ingest benchmark: metric definitions and test data

The meter-scaling benchmark of §9.3 is defined formally as follows. A run at fleet size N submits one transaction per meter per epoch over $E = 2$ epochs. Let N_{ok} denote confirmed transactions, and attribute every non-confirmed transaction to exactly one class: N_{send} (rejected at submission), N_{val} (executed, failed an oracle guard), or N_{exp} (accepted, never confirmed within τ_{conf}). Conservation holds by construction:

$$N = N_{\text{ok}} + N_{\text{send}} + N_{\text{val}} + N_{\text{exp}} \quad (23)$$

Confirmed goodput over a burst starting at t_0 with last observed confirmation at t_{last} (the value reported as TPS throughout §9):

$$\text{TPS} = \frac{N_{\text{ok}}}{t_{\text{last}} - t_0} \quad (24)$$

Per-transaction latency is $L_i = t_i^{\text{conf}} - t_i^{\text{send}}$, quantised by the status-poll period τ_{poll} (reported: median, 95th percentile, maximum). The loss decomposition of Table 4 separates the validation-rejection rate from true delivery loss:

$$\nu = \frac{N_{\text{val}}}{N}, \quad \delta = \frac{N_{\text{send}} + N_{\text{exp}}}{N} \quad (25)$$

Because resubmission is idempotent (§9.3), k independent retry rounds reduce effective delivery loss to δ^{k+1} ; the measured worst case $\delta = 0.0035$ (100,000 meters) yields $\delta^2 \approx 1.2 \times 10^{-5}$ after a single retry.

The synthetic workload assigns meter i the reading pair

$$E_{\text{prod}}(i) = 80 + (7i \bmod 420), \quad E_{\text{cons}}(i) = 40 + (3i \bmod 210) \quad (26)$$

which deliberately drives a fixed $\approx 0.47\%$ of indices past the anomaly gate (Equation 17 with $\kappa = 1000$): those transactions fail with `AnomalousReading`, at a fee, in every epoch — a deterministic, in-band probe that the oracle’s input validation continues to fire under full fleet load. The per-transaction compute figures (Table 4) are sampled from the tail of each epoch’s confirmed set, because the rotating ledger prunes older transaction history under loads above roughly 10^5 transactions.

Harness parameterisation (data):

Parameter	Value	Meaning
N	80 ... 10^5	fleet size (distinct meter PDAs)
E	2	epochs per run (init + steady-state)
epoch spacing	≥ 61 s	on-chain <code>min_reading_interval</code> + 1 (on-chain clock)
W	3,000 tx	in-flight window (unconfirmed cap)
τ_{poll}	1.5 s	bulk <code>getSignatureStatuses</code> sweep period
τ_{conf}	90 s	confirmation deadline before a send is expired
blockhash refresh	10 s	cached recent-blockhash renewal
send workers	64	concurrent submission tasks
CU sample	≤ 200 tx/epoch	tail-of-burst <code>getTransaction</code> sample
retry	none	loss figures are first-attempt delivery
commitment	confirmed	counting threshold for N_{ok}

Harness: `scripts/bench-meter-throughput.ts`; per-run artifacts under `test-results/meter-throughput-<N>m-<timestamp>.{json,md}` with the combined tables in `test-results/meter-scaling-summary.md`.

11.11 Order-entry benchmark: metric definitions and test data

The order-entry benchmark of §9.2 (Table 3, “CDA order entry”) reuses the metric definitions of §11.10 unchanged — outcome conservation (Equation 23), confirmed goodput (Equation 24), and the loss split (Equation 25) — applied to a single burst of $N = 10^4$ `submit_limit_order_sharded` transactions rather than per-meter epochs. The synthetic workload assigns order $i \in \{0, \dots, N - 1\}$ deterministically (`scripts/bench-trading-throughput.ts:166–169`):

$$s(i) = i \bmod 2, \quad q(i) = (1 + (i \bmod 50)) \cdot 10^9, \quad p(i) = 3 \cdot 10^6 + (i \bmod 100) \cdot 10^4 \quad (27)$$

alternating buy/sell side s , energy amount q sweeping 1–50 kWh in 9-decimal atoms, and limit price p sweeping 3.00–3.99 THBG/kWh in 6-decimal minor units. Every price lies inside the admission band of Equation 1, so — unlike the meter workload, which deliberately trips the anomaly gate — no order is validation-rejected by design and the expected on-chain error rate is $\nu = 0$. Each order initialises its own PDA (`[b"order", authority, order_id]` with `order_id = base + i`) and lands on zone shard

$$\sigma_{\text{ord}}(i) = i \bmod 16 \quad (28)$$

(round-robin, in contrast to the key-derived registry shard of Equation 21). The write set of order i is thus its own order PDA plus shard $\sigma_{\text{ord}}(i)$ – 16-way disjoint by construction – plus two **shared** writable accounts that serialise the path: the single fee-payer/authority, and the zone-market account, which the account context declares writable (programs/trading/src/lib.rs:1755) although the handler mutates only the per-shard account. Transport and parameterisation are identical to §11.10 ($W = 3\{, \}000$, $\tau_{\text{poll}} = 1.5$ s, $\tau_{\text{conf}} = 90$ s, 64 send workers, no retry, confirmed commitment); zone 701, 16 pre-initialised shards.

Measured outcome (data, single run, 2026-07-07):

Quantity	Value	Definition
N	10,000	orders submitted (one tx each)
N_{ok}	9,977	confirmed (Equation 23)
$N_{\text{send}} / N_{\text{val}} / N_{\text{exp}}$	0 / 0 / 23	send-rejected / on-chain error / expired
ν	0%	validation-rejection rate (Equation 25) – as designed
δ	0.23%	delivery loss (Equation 25); $\delta^2 \approx 5 \times 10^{-6}$ after one retry
$t_{\text{last}} - t_0$	55.5 s	burst wall time
TPS	179.65	confirmed goodput (Equation 24)
$L_{\text{p50}} / \text{p95} / \text{max}$	2,173 / 3,565 / 4,563 ms	send→confirmed latency ($\pm \tau_{\text{poll}}$)
CU min / med / max	9,884 / 12,886 / 23,384	per-tx compute, $n = 200$ tail sample

The base order-entry write (CU min 9,884) is the cheapest hot-path write measured – below the 13.3–13.6 k CU steady-state meter write of §11.10 – yet its confirmed goodput (180 TPS) is 2.6× lower than meter ingest at the same transaction count on the same harness (462 TPS at $N = 10^4$, Table 4). Since the transport, in-flight window, and fee-payer topology are identical, the gap isolates the one structural difference: every order additionally write-locks the shared zone-market account, so the 16 shards serialise behind it. The entire measured loss is transport ($N_{\text{val}} = 0$, $N_{\text{exp}} = 23$), recoverable by the idempotent-retry argument of §11.10.

Harness: `scripts/bench-trading-throughput.ts; run artifact test-results/trading-order-entry-100000-2026-07-07T12-58-45-123Z.{json,md}`, summarised in [BENCHMARKS.md](#) §10.

11.12 Community-month benchmark: conservation identities and test data

The month simulation of §9.4 adds a **conservation** layer on top of the delivery metrics of §11.10. Let $S_a(i, d)$ denote prosumer i ’s oracle-accepted surplus on day d – the sum of per-interval surpluses over the readings that passed the anomaly gate – and $C = 10$ kWh the daily sell cap. The lifecycle mints

$$M(i, d) = \min(S_a(i, d), C) \quad (29)$$

as GRID for each prosumer-day above the 0.1 kWh dust floor, and certifies the remainder as RECs at month end:

$$R(i) = \sum_d S_a(i, d) - \sum_d M(i, d) \quad (30)$$

Because `issue_erc` claims through the registry’s `mark_erc_claimed` CPI, the identity GRID + REC ≤ metered generation is enforced **on-chain** per meter, not merely by harness arithmetic (§11.7). The run must therefore satisfy three closure identities, each checkable from chain state:

$$\sum_{i,d} M(i, d) = E_{\text{settle}} = E_{\text{burn}}, \quad \sum_i R(i) = \frac{\text{REC supply}}{10^3}, \quad \sum_m T_m = P_{\text{sell}} + W \quad (31)$$

where E_{settle} and E_{burn} are the energy moved by settlement and retired by burns, the REC mint carries 1,000 base units per kWh, and for each match m the buyer-escrow debit $T_m = \lfloor q_m p^* / 10^9 \rfloor$ splits into seller proceeds P_{sell} and the flat wheeling charge W of Equation 8.

Measured outcome (canonical run, 2026-07-07; all values re-derived from live chain state by `scripts/audit-community-month.ts`, 15/15 assertions passing):

Quantity	Value	On-chain source
confirmed readings	229,481	Σ per-meter total_readings (80 MeterState PDAs)
anomaly rejections	919	deterministic across 7 replays; fee-paid, on-ledger
orders	2,396	Order PDA count
settled matches	353	TradeNullifier count (order nullifiers = 706 = 2 $\times$)
GRID minted (Wh)	3,290,724	Σ settled_net_generation over 12 registry meters
GRID supply after burns	68 Wh	getTokenSupply — only escrow-seed dust remains
REC certified (Wh)	4,962,778	Σ claimed_erc_generation = 12 certificates = supply / 10 ³
closure Equation 31	$3,290,724 + 4,962,778 = 8,253,502$	oracle-accepted surplus, exact
currency conservation	$13,137 = 12,832 + 305$	buyer outflow = seller proceeds + wheeling
month wall time	1,394.7 s	1858 $\times$ real-time compression

Transport and loss taxonomy are those of §11.10 with one addition: the telemetry phase packs up to $B = 10$ readings per transaction along each per-meter chain (ordering inside a transaction is free), which raises the carried reading rate to $\approx 10\times$ the transaction rate and eliminated delivery loss entirely in the canonical run ($\delta = 0$; the only non-confirmed submissions are the 919 deliberate anomaly rejections, $\nu = 0.40\%$).

Harness: `scripts/bench-community-month.ts` (dataset exporter runs against the smart-meter simulator’s Python API); canonical artifact `test-results/community-month-80m-12p-2026-07-07T16-45-40-924Z.{json,md}`, audited run ... T17-18-23-134Z, summarised in [BENCHMARKS.md](#) §11.